Task 1 - Scaffold the Enrichment Pipeline

In this task we will look at using Copilot Chat to create a new file named 'pipeline.py'

Open Copilot Chat with Ctrl+Alt+I (Windows/Linux) or Ctrl+Cmd+I (Mac), then select Agent from the agents dropdown at the bottom of the chat view. Use the following prompt, or something similar (if you want to experiment):

"Create a file named pipeline.py that imports InboundLeadInput from lead_models.py. Write a function qualify_lead(lead: InboundLeadInput) -> dict that calculates an enterprise score based on employee count and budget, and assigns a tier (Enterprise, Growth, or Startup)."

Verify that Copilot generated the logic (don't worry if you don't fully understand all the code) and maps the pydantic model correctly.

Use the command:

> python3 -c "from lead_models import InboundLeadInput; lead = InboundLeadInput(company_name='Acme Corp', employee_count=50, monthly_budget_usd=5000, contact_email='lead@acme.com'); print('SUCCESS:', lead)"

If it prints the model successfully, the mapping has been done correctly.

We also want to check that if we use invalid data, it doesn't work. Use the command:

> python3 -c "from lead_models import InboundLeadInput; InboundLeadInput(company_name='Acme Corp', employee_count=-5, monthly_budget_usd='not_a_number', contact_email='lead@acme.com')"

The model should immediately detect incorrect data and give a validation error

If you open up a terminal within VSCode, you can run your Python code (make sure you are in the project root folder) by running the command:

python3 pipeline.py or py pipeline.py

Task 2: Build a Deterministic Scoring Tool

In this task, we will create a separate scoring_tools.py file using the Copilot Agent to handle the math logic rather than leaving it inside the main pipeline script.

In the Composer, use the prompt:

> "Create scoring_tools.py containing a function calculate_tier_multiplier(budget: int, headcount: int) -> float. It must apply a deterministic formula: budget divided by headcount, returning a qualification multiplier."

This prompt should make sure the LLM (Large Language Model) doesn't do any guesswork, as we explicitly tell Copilot to write a hard-coded formula that is repeatable, as well as ensuring type safety since we defined the types of data as well.

Check the file created and ensure you are happy with the code (again, do not worry if you don't fully understand the code)

Now you will update the pipeline.py file and verify that the newly created file is being called.

1. Open pipeline.py in your VSCode editor.
2. Highlight the section of code where you want to calculate the score or tier (typically inside or right before your qualify_lead function).
3. Trigger the Inline Edit by pressing:
4. Windows/Linux: Ctrl + I
5. A small chat input box will appear directly over your code. Type this prompt into it:
6. "Import calculate_tier_multiplier from scoring_tools and update this function to use it for the lead scoring logic."
7. Copilot will analyze your code inline, write the import statement at the top of pipeline.py (if it isn't already there), and replace the old math logic with a call to your new scoring_tools.py function.
8. Press Keep to accept the changes, or Undo (Esc) if you want to try again.

Once accepted, check that pipeline.py now successfully imports from scoring_tools and calls calculate_tier_multiplier(lead.monthly_budget_usd, lead.employee_count), proving your multi-file factory modules are communicating properly!

Task 3: Implement a Fraud & Bot Circuit Breaker

In this last task, you will now implement some security safeguards to protect the sales pipeline from low-quality/fake submissions.

In this next step, where you will write your prompt, this can be done in Copilot Agent or Ask mode. Generally, we use Copilot Agent for Multi-file creation, large refactoring, etc. We use Ask mode for quick questions, debugging specific pieces of code, or planning things out. In this scenario, you can use either.

Prompt to use in Copilot Agent (Ctrl+Alt+I / Ctrl+Cmd+I, Agent dropdown) or Ask mode:

> "Update pipeline.py to include a circuit breaker check for disposable email domains (e.g., checking if the email contains strings like 'mailinator', 'tempmail', or 'test.com'). If a disposable domain is detected, trip the circuit breaker, halt execution, and flag the lead as 'Rejected-Spam'."

Now let's verify that our circuit breaker works!

1. Open a terminal in VSCode where your project files are (You should already be here, but just check the path in the terminal to make sure)
2. Execute a Python snippet that passes a mock lead containing a blocked/disposable email domain
    - python3 -c "from pipeline import qualify_lead; from lead_models import InboundLeadInput; lead = InboundLeadInput(company_name='Spam Corp', employee_count=10, monthly_budget_usd=1000, contact_email='fake@mailinator.com'); qualify_lead(lead)"
3. Check the output to confirm the circuit breaker intercepted the request

You will know the circuit breaker worked successfully if the script halts execution or prints a safety warning/rejection message (e.g., "Circuit Breaker Triggered: Disposable email detected") instead of processing the lead through the scoring pipeline.

If you finished early, or just want some extra practice outside of the session, building on the earlier tasks, here are some optional additional add-ons you can try out!

Option 1: The Self-Correction Loop (Agentic Retry)

What it adds: Instead of the circuit breaker just immediately throwing an error or rejecting a bad lead, have the agent attempt to fix or enrich the data first.
Use the prompt below:
"Update pipeline.py so that if a lead fails validation or has missing parameters, the agent attempts to infer or clean the data twice before tripping the circuit breaker."

This task is to show you how you can move from a rigid, linear code to a more agentic loop format (try, fail, reflect, retry)

Option 2: Add a Second Specialized Tool File

What it adds: Moving from a 2-file architecture to a true multi-file factory.
Use the prompt below:
"Create a new file called enrichment_tools.py that takes a company name, looks it up against a mocked database dictionary, and returns an estimated annual revenue bracket. Integrate this tool into pipeline.py."

This task is to show you how to keep your factory modular - by separating math logic, data contracts, and orchestration into clean, isolated files.

Option 3: Write an Automated Test Suite via GitHub Copilot

What it adds: Testing the safety guardrails.
The Prompt for Delegates:
"Create a file called test_pipeline.py that uses pytest to test our circuit breaker. Write three test cases: one valid enterprise lead, one startup lead, and one fake lead with a disposable 'mailinator.com' email to prove the circuit breaker catches it.

This task helps demonstrate that AI-generated code can be immediately verified with automated testing, right within VSCode using Copilot! And yes, it is always essential to verify and test your code, whether it's AI-generated or not!